

Vektoripaja

Paperinen työpaketti

Low-poly 3D-mallinnin Windows-työpöytäsovelluksena (Python): Inkscapen SVG → 3D-malli → .obj.

Tekoälyavusteinen työtapa: Copilot suunnittelee, GitHub Copilot toteuttaa.

Vk 40/2026–vk 9/2027 · syysloma vk 42 · joululoma 18.12.–10.1. · talviloma vk 8 · luovutus pe 5.3.2027

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Repository on julkinen. Sinne ei laiteta henkilötietoja, testaajien nimiä eikä itsearviointia. Ne lähetetään ohjaajalle Teamsissa.

Viikkopalaveri ohjaajan kanssa on maanantaina tai tiistaina. Päivä sovitaan viikolla 40.

Aikataulu

Vko	Pvm	Viikon aihe	Vaihe
40	28.9.– 2.10.	Aloitus: työkalut, repository ja oma MVP	A
41	5.–9.10.	Harjoitussykli: pyörivä kuutio julki	A
42	12.– 16.10.	Syysloma – ei projektityötä	–
43	19.– 23.10.	SVG-tuonti: polut näkyviin	B
44	26.– 30.10.	Hierarkia: layerit ja ryhmät osiksi	B
45	2.–6.11.	Revolve: profiilista pyörähdyskappale	B
46	9.–13.11.	Inflate ja kamera: putki ja suorat näkymät	B
47	16.– 20.11.	Transformit: valitse, siirrä, kierrä ja skaalaa	B
48	23.– 27.11.	Pivot: kiertopiste prosentteina	C

49	30.11.– 4.12.	OBJ-vienti: osat erillisinä Blenderiin	C
50	7.–11.12.	Tallennus ja MVP: versio v0.1 julki	C
51	14.– 17.12.	Katselmointi: asiakkaat kokeilevat MVP:tä	C
52	21.– 25.12.	Joululoma – ei projektityötä	–
53	28.12.– 1.1.	Joululoma – ei projektityötä	–
1	4.–8.1.	Joululoma – ei projektityötä	–
2	11.–15.1.	Paluuviikko: testit läpi ja P1- järjestys	D
3	18.–22.1.	Päivitä SVG: muokattu piirros malliksi	D
4	25.–29.1.	Jousto ja P1: rästit tai seuraava toiminto	D
5	1.–5.2.	Saavutettavuus: isot kahvat ja näppäimistö	D
6	8.–12.2.	Julkaisutesti: julkaisuehdokas v1.0-rc1	E

7	15.–19.2.	v1.0: julkaisu ja asiakkaiden vahvistus	E
8	22.–26.2.	Talviloma – ei projektityötä	–
9	1.–5.3.	Näyttö: demo ja luovutus	E

Palautus viimeistään pe 5.3.2027. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe A – Työkalut ja harjoitussykli

Vko 40 · 28.9.–2.10. – Aloitus: työkalut, repository ja oma MVP

VS Code ja GitHub ovat korkean kontrastin teemassa. Komennot `git --version` ja `python --version` tulostavat versiot. Julkinen repository on kloonattu VS Codeen ja jaettu ohjaajalle. `project-docs/mvp.md` on pushattu.

- Tarkista, mitä on jo asennettu. Asenna näistä puuttuvat: VS Code, Git, Python 3.13, VS Coden Python-laajennus, Inkscape ja Blender. Vaihda VS Codeen ja GitHubiin korkean kontrastin teema.
- Luo julkinen repository README-tiedoston kanssa. Kloonaa se VS Codeen. Jaa se ohjaajalle.
- Kirjoita MVP omin sanoin tiedostoon `project-docs/mvp.md`. Tee commit ja push. Lähetä kysymyslista asiakkaille.
- Mittaa yhden agenttipyyntöä hinta viestipohjalla. Kirjaa hinta päiväkirjaan.

Valmis kun: Ohjaaja näkee repositoryn, `mvp.md` on GitHubissa, kysymyslista on lähetetty, ja päiväkirjassa on agenttipyyntöä hinta tai merkintä "odottaa käyttönäkymää".

Työnäyte Git-repositoryyn ennen rastia: Näytä: repositoryn osoite, project-docs/mvp.md GitHubissa ja agenttipyyntö hinta päiväkirjassa.

Vko 41 · 5.–9.10. – Harjoitus sykli: pyörivä kuutio julki

Sovelluksen ikkunassa pyörii kuutio. Testitapaukset T01 ja T02 menevät läpi. GitHubin releasessa on zip, jonka itsetesti meni läpi.

- Päätä, hyväksytkö teknisen ehdotuksen. Perustele päätös suunnitelmaan. Pura pohja. Luo virtuaaliympäristö ja asenna kirjastot itse komennoilla.
- Kirjoita testitapaus T01 ennen koodia. Mitä odotat, kun kiertokulma saa ajan 0 ja nopeuden 0,5? Entä kun aika on 3?
- Tee kortti #1 (kuutio) työsyklillä mallikortin avulla. Aja T01.
- Tee kortti #2 (release) työsyklillä. Aja T02 releasen zipistä puretulla sovelluksella.

Valmis kun: Releasen zipistä purettu sovellus käynnistyy, ja kuutio pyörii. Actionsin ajo on vihreä, eli itsetesti meni läpi. `pytest` näyttää, että T01 menee läpi. Issuet #1 ja #2 on suljettu commitilla.

Työnäyte Git-repositoryyn ennen rastia: Näytä: releasen osoite, issueet #1 ja #2 suljettuina sekä T01:n

ja T02:n tulokset issueissa.

Vko 42 · 12.–16.10. – Syysloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 43 tilatiedoston Seuraavana-kohdasta.

Vaihe B – MVP-ydin: SVG:stä muotoja

Vko 43 · 19.–23.10. – SVG-tuonti: polut näkyviin

Inkscapessa piirretty SVG avautuu sovellukseen. Sen polut näkyvät 3D-näkymässä viivoina.

- Piirrä Inkscapessa testitiedosto, jossa on nimetyt layerit ja kolme sisäkkäistä ryhmää.
- Kirjoita testitapaukset T03–T05 ennen toteutusta: kolme polkua, SVG, jossa on `<script>`, ja tiedosto, joka ei ole SVG.
- Kirjaa kansiorakenne ja moduulien rajat suunnitelmaan.
- Toteuta tuonti työsyklillä kaistalla A, tiedosto kerrallaan.
- Kokeile SVGLoaderin rajoitteita omalla tiedostollasi. Kirjaa ne kirjastot.md:hen.

Valmis kun: Oma SVG avautuu tiedostonvalinnalla, T03–T05 menevät läpi, ja `project-docs/kirjastot.md:ssä` on kaksi SVGLoaderin rajoitetta omalla tiedostolla kokeiltuna.

Työnäyte Git-repositoryyn ennen rastia: Näytä: oma SVG-tiedosto, T03–T05 issueissa ja project-docs/kirjastot.md.

Vko 44 · 26.–30.10. – Hierarkia: layerit ja ryhmät osiksi

SVG:n layerit ja ryhmät muuttuvat scene graphiksi eli 3D-näkymän vanhempi–lapsi-puuksi. Hierarkiapaneeli näyttää osien nimet sisennettyinä.

- ☐ Kirjoita hierarkian muunnoksen rajapinta ja testitapaukset T06–T08 ennen toteutusta.
- ☐ Toteuta muunnos puhtaana funktiona ja hierarkiapaneeli kaistalla C.
- ☐ Päätä valinnan toiminta. Kirjaa se suunnitelmaan.
- ☐ Vertaa kahta tapaa valita revolve. Tuo suositus ensi viikon palaveriin.

Valmis kun: Oman tiedostosi kolme ryhmätasoa näkyvät paneelissa sisennettyinä, T06–T08 menevät läpi, ja suunnitelmassa on valinnan toiminta perusteluineen.

Työnäyte Git-repositoryyn ennen rastia: Näytä: rajapinta ja T06–T08 issueissa, hierarkiapaneeli Pages-osoitteessa ja valinnan päätös suunnitelmassa.

Vko 45 · 2.–6.11. – Revolve: profiilista

pyörähdyskappale

Puoliprofiilista syntyy pyörähdyskappale. Segmenttien määrää voi säätää välillä 3–32. Kappaletta voi kiertää hiirellä joka puolelta.

- Sovi palaverissa revolverin valintatapa. Kirjaa, mitä profiilille tehdään akselin väärällä puolella.
- Kirjoita testitapaukset T09–T11 ennen toteutusta. Kysy tarvittaessa kysymystilasta, miten LatheGeometry lukee pisteet.
- Toteuta revolve, segmenttisäädin ja orbit työsyklillä.

Valmis kun: Oma profiilisi pyörähtää kappaleeksi, segmenttisäädin muuttaa särmien määrää välillä 3–32, ja T09–T11 menevät läpi.

Työnäyte Git-repositoryyn ennen rastia: Näytä: T09–T11 issueissa, pyörähdyskappale Pages-osoitteessa ja sovittu valintatapa issue-kommenttina.

Vko 46 · 9.–13.11. – Inflate ja kamera: putki ja suorat näkymät

Viivapolusta syntyy putki, jossa on 3–8 sivua.

Orientaatiokuutiolla kamera kääntyy katsomaan suoraan edestä, sivulta tai ylhäältä. View lock -painike lukitsee kierron.

- Kirjoita testitapaukset T12–T14 ennen toteutusta.

- Toteuta inflate ja sivumäärän säädin työsyklillä. Kirjaa TubeGeometryn rajoitteet.
- Toteuta orientaatiokuutio ja view lock -painike työsyklillä.

Valmis kun: Inflate on valmis, kun oma polkusi muuttuu putkeksi, sivumäärän voi valita väliltä 3–8 ja T12–T13 menevät läpi. Kamera on valmis, kun kuution sivua klikkaamalla kamera kääntyy suoraan akselin suuntaan, view lock estää kierron ja T14 menee läpi. Jos kamera siirtyy viikolle 47, se on sovittu vaihtoehto eikä virhe.

Työnäyte Git-repositoryyn ennen rastia: Näytä: T12–T14 issueissa, putki ja suorat näkymät Pages-osoitteessa ja TubeGeometryn rajoitteet kirjastot.md:ssä.

Vko 47 · 16.–20.11. – Transformat: valitse, siirrä, kierrä ja skaalaa

Osan voi valita, siirtää, kiertää ja skaalata. Lapset seuraavat vanhempaa.

- Kirjoita testitapaus T15 ennen toteutusta. Mitä odotat lapsen paikalle, kun vanhempaa siirretään 10 yksikköä?
- Toteuta valinta suunnitelmasi mukaan. Toteuta siirto, kierto ja skaalaus työsyklillä.
- Aja T15. Jos se ei mene läpi kahdella yrityksellä, avaa havaintoissue.

Valmis kun: Osan voi valita suunnitelmasi mukaan.

Siirto, kierto ja skaalaus toimivat. T15 menee läpi, tai siitä on havaintoissue.

Työnäyte Git-repositoryyn ennen rastia: Näytä: T15
issueissa, transformit Pages-osoitteessa ja P0-tilanne
palaverin issue-kommentissa.

Vaihe C – MVP valmiiksi ja katselmointi

Vko 48 · 23.–27.11. – Pivot: kiertopiste prosentteina

Pivot on oletuksena kappaleen keskipisteessä. Sen voi asettaa transformivalikossa prosentteina X/Y/Z.

- ☐ Kirjoita pivotin syötteen tarkistuksen rajapinta ja testitapaukset T16–T18 ennen toteutusta.
- ☐ Toteuta pivot ja syötteen tarkistus työsyklillä.
- ☐ Lisää transformivalikko prosenttisyötöllä käyttöliittymävaatimuksesi mukaan.

Valmis kun: Pivot 50/50/50 on keskipisteessä, prosentit muuttavat pivotin paikkaa, virheellinen syöte käsitellään kriteerisi mukaan, ja T16–T18 menevät läpi.

Työnäyte Git-repositoryyn ennen rastia: Näytä: rajapinta ja T16–T18 issueissa sekä transformivalikko Pages-osoitteessa.

Vko 49 · 30.11.–4.12. – OBJ-vienti: osat erillisinä Blenderiin

Malli tallentuu .obj-tiedostoksi, jossa jokainen osa on oma objektinsa. Tiedosto aukeaa Blenderissä.

- ☐ Kirjoita objektijaon rajapinta ja testitapaukset T19–T20 ennen toteutusta.

- ☐ Toteuta vienti niin, että osien nimet säilyvät.
- ☐ Laske o-rivit VS Codessa. Avaa tiedosto Blenderissä kuvaohjeen mukaan.
- ☐ Kirjoita ensimmäinen virheenkorjausketju havaintoissueistasi. Kirjoita sille regressiotesti.

Valmis kun: Kolmen osan mallista syntyy .obj-tiedosto, jossa on kolme o-riviä osien nimillä. Tiedosto aukeaa Blenderissä. T19–T20 menevät läpi.

Virheenkorjausketjun kuusi osaa ovat havaintoissuessa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: T19–T20 issueissa, viety .obj-tiedosto, kuva Blenderistä ja virheenkorjausketju 1 havaintoissuessa.

Vko 50 · 7.–11.12. – Tallennus ja MVP: versio v0.1 julki

Projektin voi tallentaa JSON-tiedostoksi ja avata samassa tilassa. MVP on julkaistu versiona v0.1.

- ☐ Vertaa tallennustapoja oman suunnitelmasi kriteereillä. Kirjaa valinta suunnitelmaan.
- ☐ Kirjoita testitapaukset T21–T22 ennen toteutusta.
- ☐ Toteuta tallennus ja avaus työsyklillä. Kirjoita tietoturva-arvio tuonnista ja avauksesta.
- ☐ Julkaise MVP. Merkitse versio tagilla v0.1.

Valmis kun: Tallennettu projekti avautuu samassa tilassa, rikottu tiedosto antaa virheilmoituksen, `project-docs/tietoturva.md` on repositoryssa, ja tagi `v0.1` näkyy GitHubissa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: tallennusvertailu suunnitelmassa, T21–T22 issueissa, `project-docs/tietoturva.md` ja tagi `v0.1`.

Vko 51 · 14.–17.12. – Katselmointi: asiakkaat kokeilevat MVP:tä

Asiakkaat ovat kokeilleet MVP:tä. Jokaisesta havainnosta on issue, jossa on asiakkaan antama prioriteetti.

- ☐ Valmistele viiden minuutin demo omalla Inkscape-tiedostolla.
- ☐ Pidä katselmointi asiakkaiden kanssa. Pidä heidän havaintonsa erillään omasta tulkinnastasi.
- ☐ Tee havainnoista issuet. Kirjaa niihin asiakkaan prioriteetti.
- ☐ Kirjoita tilatiedoston Seuraavana-kohtaan viikon 2 ensimmäinen tehtävä issue-numeroineen.

Valmis kun: Katselmointiloki on tiedostossa `project-docs/katselmointi.md`, jokaisesta havainnosta on issue prioriteetteineen, ja tilatiedoston Seuraavana-

kohdassa on viikon 2 ensimmäinen tehtävä issue-numeroineen.

Työnäyte Git-repositoryyn ennen rastia: Näytä: project-docs/katselmointi.md, havaintoissuet prioriteetteineen ja tilatiedoston Seuraavana-kohta.

Vko 52 · 21.–25.12. – Joululoma

Joululoma alkaa pe 18.12. Ei projektityötä eikä korvaavia tehtäviä.

Vko 53 · 28.12.–1.1. – Joululoma

Joululoma jatkuu. Ei projektityötä eikä korvaavia tehtäviä.

Vko 1 · 4.–8.1. – Joululoma

Joululoma päättyy su 10.1. Jatka ma 11.1. viikolla 2 tilatiedoston Seuraavana-kohdasta.

Vaihe D – Jatkokehitys katselmoinnin pohjalta

Vko 2 · 11.–15.1. – Paluuviikko: testit läpi ja P1-järjestys

Kaikki testit menevät läpi. P1-järjestys on sovittu tuntiarvioineen. Yksi pieni korjaus katselmoinnista on GitHubissa.

- Aja `npm install`, `npm run dev` ja `npm test`. Kirjaa tulos.
- Ehdota P1-järjestys ja tuntiarviot asiakkaiden prioriteettien pohjalta. Sovi järjestys palaverissa.
- Tee yksi pieni korjaus katselmoinnin havainnoista kaistalla B.

Valmis kun: `npm test` näyttää, että kaikki testit menevät läpi, P1-järjestys on sovittu palaverissa, ja korjauksen commit on GitHubissa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: testiajon tulos päiväkirjassa, P1-järjestys issueissa ja korjauksen commit.

Vko 3 · 18.–22.1. – Päivitä SVG: muokattu piirros malliksi

Päivitä SVG -toiminto rakentaa mallin uudelleen muokatusta tiedostosta. Osien transformit säilyvät.

- Sovi palaverissa, miten osat tunnistetaan uudesta tiedostosta.
- Kirjoita testitapaus T23 ennen toteutusta.
- Toteuta toiminto omassa haarassa. Aja kaikki testit. Liitä haara päähaaraan pull requestilla.

Valmis kun: Muokattu SVG päivittyy malliin, transformit säilyvät, kaikki testit menevät läpi, ja pull request on yhdistetty päähaaraan.

Työnäyte Git-repositoryyn ennen rastia: Näytä: T23
issuessa ja yhdistetty pull request.

Vko 4 · 25.–29.1. – Jousto ja P1: rästit tai seuraava toiminto

Jos P0-rästejä on, ne ovat valmiit. Muuten seuraava sovittu P1-toiminto toimii.

- Tee ensin P0-rästit omassa haarassa, jos niitä on.
- Kirjoita seuraavan P1-toiminnon testitapaus T24 ennen toteutusta.
- Toteuta toiminto työsyklillä. Kirjoita toinen virheenkorjausketju.

Valmis kun: P0-rästit on liitetty päähaaraan pull requestilla, tai P1-toiminto toimii ja T24 menee läpi. Virheenkorjausketjun kuusi osaa ovat havaintoissuessa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: T24
issuessa, valmis toiminto Pages-osoitteessa ja
virheenkorjausketju 2 havaintoissuessa.

Vko 5 · 1.–5.2. – Saavutettavuus: isot kahvat ja näppäimistö

Sovelluksessa on isot kahvat ja näppäimistökäyttö.
Lighthousen tulokset ennen ja jälkeen on kirjattu.
Mikään kohta ei heikentynyt, ja löydetyt puutteet on
korjattu.

- ☐ Aja Lighthouse. Kirjaa perusmittaus.
- ☐ Tarkista käyttöliittymävaatimuksesi. Kirjoita testitapaukset T25–T26 ennen toteutusta.
- ☐ Toteuta isot kahvat ja näppäimistökäyttö työsyklillä.
- ☐ Aja T25–T26 ja Lighthouse uudelleen. Vertaa tuloksia.

Valmis kun: Lighthousen tulokset ennen ja jälkeen on kirjattu, mikään kohta ei heikentynyt, T25–T26 menevät läpi, ja löydetyt puutteet on korjattu tai kirjattu issueiksi.

Työnäyte Git-repositoryyn ennen rastia: Näytä:
Lighthousen tulokset ennen ja jälkeen, T25–T26
issueissa ja korjausten commitit.

Vaihe E – Julkaisu ja näyttö

Vko 6 · 8.–12.2. – Julkaisutesti: julkaisuehdokas v1.0-rc1

Julkaisuehdokas v1.0-rc1 on jäädytetty. Toinen opiskelija on kulkenut README:n avulla koko polun Inkscape-piirroksesta .obj-tiedostoon.

- Jäädytä sisältö. Merkitse julkaisuehdokas tagilla v1.0-rc1.
- Täydennä README ja käyttöohje. Kerro ohjeessa Inkscapen nimeämissäännöt.
- Pyydä julkaisutestaajaa kulkemaan koko polku puhtaassa ympäristössä. Pidä hänen havaintonsa erillään omasta tulkinnastasi.

Valmis kun: Julkaisutestaajan .obj-tiedosto aukeaa, ja epäröinnit on listattu pöytäkirjaan. Testaajan nimi ja sanatarkat lausumat on lähetetty ohjaajalle Teamsissa. Repositoryyn on kirjattu vain rooli.

Työnäyte Git-repositoryyn ennen rastia: Näytä: tagi v1.0-rc1, README ja käyttöohje sekä project-docs/julkaisutesti.md.

Vko 7 · 15.–19.2. – v1.0: julkaisu ja asiakkaiden vahvistus

Julkaisutestin estävät havainnot on korjattu. Versio v1.0 on julki. Asiakkaat ovat vahvistaneet, että se toimii heidän koneellaan. Näyttömatriisin jokaisella rivillä on linkki työnäytteeseen.

- Korjaa estävät havainnot. Kirjoita kolmas virheenkorjausketju.
- Julkaise v1.0 tagilla. Pyydä asiakkaita vahvistamaan, että se toimii heidän koneellaan.
- Linkitä näyttömatriisin rivit päiväkirjan tunnusten avulla.

Valmis kun: Estävät issuet on suljettu, tagi v1.0 näkyy GitHubissa, asiakkaat ovat vahvistaneet toimivuuden, ja jokaisella matriisin rivillä on linkki.

Työnäyte Git-repositoryyn ennen rastia: Näytä: tagi v1.0, virheenkorjausketju 3, asiakkaiden vahvistus ja linkitetty näyttömatriisi.

Vko 8 · 22.–26.2. – Talviloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 9 näytön valmistelulla.

Vko 9 · 1.–5.3. – Näyttö: demo ja luovutus

Ajastettu 8–10 minuutin demo on pidetty. Kaikki viisi osaa on näytetty. Aineisto on luovutettu pe 5.3.2027.

- Jäädytä sisältö maanantaina. Tarkista, että jokainen työnäyte on repositoryssa.
- Harjoittele demo ajastettuna: tuotos, yksi tekninen ratkaisu omin sanoin, korjattu virhe, Git-historia ja työnkulku yhdellä kortilla.
- Kirjoita itsearviointi kolmesta tilanteesta. Lähetä se ohjaajalle Teamsissa.

Valmis kun: Demo on pidetty 8–10 minuutissa, kaikki viisi osaa on näytetty, itsearviointi on lähetetty ohjaajalle, ja aineisto on luovutettu perjantaina.

Työnäyte Git-repositoryyn ennen rastia: Näytä: demo, linkitetty näyttömatriisi ja itsearvioinnin lähetyspäivä päiväkirjassa.

Viimeiset viisi päivää

Ma 1.3. · Sisältöjäädytys: työnäytteet repositoryssa

Ti 2.3. · Aineisto: päiväkirja, AI-loki ja näyttömatriisin linkit

Ke 3.3. · Demon harjoittelu ajastettuna toiselle henkilölle

To 4.3. · Puskuri ja itsearviointi Teamsissa ohjaajalle

Pe 5.3. · Demo ja luovutus

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

Copilot vko 40	Microsoft 365 Copilot. Tekoäly selaimessa BC:n tunnuksella. Suunnittelee ja pilkkoo viikon tavoitteen tehtäväkorteiksi. Tässä projektissa sana Copilot tarkoittaa aina tätä.
GitHub Copilot vko 40	koodityökalu VS Codessa. Tekoäly, joka toteuttaa tehtäväkortin VS Codessa. Tässä projektissa sana GitHub Copilot tarkoittaa aina tätä.
MVP vko 40	Minimum Viable Product. Pienin käytettävä versio. Tässä projektissa MVP on sama kuin P0 ja valmistuu ennen joulua.
P0 vko 40	Pakollinen ydin. Toiminnot, joiden pitää valmistua ennen joulua. Ilman niitä tuotosta ei ole.

P1 vko 40	Tärkeä jatko. Toiminnot, jotka tehdään joululoman jälkeen, kun P0 toimii.
P2 vko 40	Jatkolista. Toiminnot, jotka jäävät tämän näytön jälkeen tehtäviksi. Ne eivät ole hylättyjä.
SVG vko 40	vektorikuva. Kuvan tiedostomuoto, jossa kuva koostuu poluista. Inkscape tallentaa piirrokset SVG-tiedostoiksi.
low-poly vko 40	3D-malli, jossa on vähän monikulmioita. Pinnat näkyvät tasaisina, kulmikkaina paloina.
tulkki vko 40	Python-tulkki. Ohjelma, joka ajaa Python-koodin. Komento <code>python</code> käynnistää tulkin. Tässä projektissa käytetään Python 3.13:a.

pip vko 40	Pythonin paketinhallinta. Työkalu, joka asentaa Python-kirjastoja. Se tulee Pythonin mukana. Kirjastot asennetaan komennolla <code>pip install</code> virtuaaliympäristöön.
virtuaaliympäristö vko 40	.venv. Projektin oma Python-kansio <code>.venv</code> . Projektin kirjastot asennetaan sinne, joten ne eivät sekoitu koneen muihin Python-projekteihin. Kun virtuaaliympäristö on päällä, terminaalin rivin alussa lukee <code>(.venv)</code> . Se luodaan viikolla 41.
repository vko 40	Git-repository. Projektin kansio GitHubissa. Siellä on koodi, historia ja issuet. Kaikki työnäytteet ovat repositoryssa.
README vko 40	Repositoryn etusivun ohje. Kertoo, mikä työkalu on ja miten sen saa käyttöön.

kloonaus vko 40	Repositoryn kopiointi omalle koneelle. VS Code ja GitHub Copilot käsittelevät kloonattua kansiota.
commit vko 40	Yksi nimetty muutos Gitissä. Commitilla on viesti ja tunnus.
push vko 40	Commitien lähetys omalta koneelta GitHubiin.
Collaborator vko 40	Henkilö, jolle repository on jaettu. Ohjaaja näkee ja voi kommentoida repositoryä.
tilatiedosto vko 40	PROJEKTIN-TILA.md. Tiedosto, jossa on kohdat Valmista, Seuraavana, Tehdyt päätökset ja Avoimet kysymykset. Liität sen Copilotiin. Älä anna sitä GitHub Copilotille.
viestipohja vko 40	Valmis viesti, jonka kopioit ja täydennät. Sivun Kopioi-nappi kopioi sen.

krediitti vko 40	GitHub Copilotin käyttöraha. 200 krediittiä kuukaudessa. Yksi krediitti on 0,01 USD. Agenttitila kuluttaa niitä. Täydennykset ovat ilmaisia. Budjetti nollautuu kuun 1. päivänä, eikä käyttämätön osuus siirry.
agenttitila vko 40	Agent. GitHub Copilotin tila, joka muokkaa useaa tiedostoa. Kuluttaa krediittejä.
käyttönäkymä vko 40	GitHubin sivu, jolla krediittien kulutus näkyy.
projektipäiväkirja vko 40	Tämän sivun viikkokirjaus: mitä teit, miksi ja missä työnäyte on. Ladataan kansioon project-docs joka viikko.
AI-loki vko 40	Tämän sivun loki tekoälyn käytöstä: mihin pyysit apua, hyväksyitkö, korjautitko vai hylkäsitkö ja miksi.
viikkopalaveri vko 40	Viikon tapaaminen ohjaajan kanssa maanantaina tai tiistaina. Tiimi tarkoittaa tässä projektissa sinua ja ohjaajaa.

työsykli vko 41	kuusi askelta. Suunnittele, Siirrä, Rakenna, Tarkista, Raportoi ja Kirjaa. Sama jokaisessa tehtäväkortissa.
tehtäväkortti vko 41	Yksi rajattu tehtävä: tavoite, kaista, tiedostot, Älä tee -kohta, hyväksymiskriteerit ja testi.
hyväksymiskriteeri vko 41	Havaittava ehto, jonka pitää toteutua, jotta kortti on valmis. Esimerkiksi: kuutio näkyy ja pyörii.
issue vko 41	GitHub-issue. Tehtäväkortti GitHubissa, label tehtäväkortti. Suljetaan commit-viestin rivillä <code>Closes #N</code> .
mallikortti vko 41	Valmis esimerkkikortti, johon omaa korttia verrataan. Viikolla 41 siinä on myös testipohja.

kaista vko 41	A, B tai C. Toteutustapa. A: Copilot kirjoittaa koodin yksi tiedosto kerrallaan. B (viikolta 43): kommentti ja täydennys VS Codessa, myös testit. C (viikolta 44): GitHub Copilotin agenttitila useaan tiedostoon.
T01 vko 41	Testitapauksen tunnus. T tarkoittaa testitapausta. Numero yksilöi sen: T01 on ensimmäinen, T02 toinen. Odotettu tulos kirjataan, ennen kuin koodi tehdään. Lisätesti on ylimääräinen testi ilman numeroa.
tekninen pohja vko 41	PySide6, PyVista, trimesh ja pytest. Sovelluksen perusta. PySide6 (Qt) tekee ikkunan ja painikkeet. PyVista piirtää 3D-näkymän. trimesh tekee 3D-kappaleet ja .obj-tiedoston. pytest ajaa testit.

requirements.txt vko 41	kirjastolista. Tiedosto, jossa ovat projektin kirjastot versioineen. Kirjastot asennetaan komennolla <code>pip install -r requirements.txt</code> . Uusi kirjasto lisätään tänne vain ohjaajan luvalla.
tagi vko 41	Nimetty versio Gitissä, esimerkiksi v0.0.1, v0.1 tai v1.0. Tagin push käynnistää julkaisun.
GitHub Actions vko 41	GitHubin automaatio. Kun pushaat tagin, automaatio ajaa testit ja tekee Windows-version. Sitten se ajaa Windows-versiolle itsetestin ja lisää zipin releaseen.
paketointi vko 41	Sovellus ja sen kirjastot kootaan kansioksi, joka toimii ilman Pythonia. Tässä projektissa paketoinnin tekee PyInstaller tiedoston <code>vektoripaja.spec</code> ohjeilla.

itsetesti vko 41	Sovelluksen oma tarkistus: Vektoripaja.exe -- itsetesti. Se lukee esimerkki-SVG:n ja tekee siitä 3D-kappaleet ja .obj-tiedoston. Ikkunaa ei avata. GitHub Actions ajaa sen jokaiselle julkaisulle.
release vko 41	julkaisusivu. GitHubin sivu, josta valmiin version voi ladata. Tässä projektissa releasessa on zip, jossa on Windows-versio.
zip vko 41	pakattu kansio. Yksi tiedosto, jonka sisällä on kansio. Pura zip ennen käyttöä: hiiren oikea painike ja Pura kaikki.
main vko 41	päähaara. Repositoryn päälinja. Julkaisu tehdään päähaarasta.
kahden yrityksen sääntö vko 41	Kun sama asia epäonnistuu kahdesti, lopetetaan yrittäminen ja palataan Copilotille.

täydennys vko 43	GitHub Copilotin harmaa koodiehdotus VS Codessa. Hyväksytään Tab-näppäimellä. Ei kuluta krediittejä.
moduuli vko 43	Kansio tai tiedosto, jolla on yksi vastuu. Esimerkiksi tuonti on eri moduuli kuin vienti.
havaintoissue vko 43	label havainto. Issue, johon kirjataan virhe: mitä odotit, mitä tapahtui ja miten virheen saa toistettua. Suljetaan korjauscommitilla.
layer vko 43	Inkscapen taso. Inkscapen piirroksen taso. Layerin nimestä tulee mallin osan nimi.
scene graph vko 44	vanhempi–lapsi-puu. 3D-näkymän rakenne, jossa osat ovat vanhempi–lapsi-suhteessa. Kun vanhempi liikkuu, lapset liikkuvat mukana.
hierarkiapaneeli vko 44	Sovelluksen paneeli, joka näyttää scene graphin osat sisennettyinä.

puhdas funktio vko 44	Funktio, joka ei muuta mitään itsensä ulkopuolella. Sama syöte antaa aina saman tuloksen.
rajapinta vko 44	Funktion nimi, syöte ja paluuarvo. Rajapinta kirjoitetaan korttiin ennen toteutusta.
revolve vko 45	pyörähdyskappale. Puoliprofiili pyörähtää akselin ympäri, ja siitä syntyy kappale, esimerkiksi maljakko.
orbit vko 45	Näkymän kierto hiirellä mallin ympäri.
kysymystila vko 45	Ask. GitHub Copilotin tila, joka vastaa kysymyksiin mutta ei muuta tiedostoja.
inflate vko 46	polusta putki. Viivapolusta tulee putki, jossa on 3–8 sivua. Esimerkiksi johto tai sarvi.
orientaatiokuutio vko 46	Orientation Gizmo. Kuutio näkymän kulmassa. Kun klikkaat sen sivua, kamera kääntyy katsomaan mallia suoraan sivun suunnasta.

suora näkymä vko 46	Katsot mallia tasan yhden akselin suunnasta, kuin pöydän reunalta tai ylhäältä. Suunnitelmassasi tämä on osa Orthographic Plane Snapia.
ortografinen näkymä vko 46	P1. Näkymä, jossa kaukana olevat osat eivät pienene kuten valokuvassa. Tehdään P1:ssä. P0:ssa kaukana olevat osat näyttävät pienemmiltä, ja se on oikein.
view lock vko 46	Painike, joka estää kameran kiertymisen, kun teet tarkkaa työtä.
transformi vko 47	Osan siirto, kierto ja skaalaus.
pivot vko 48	Piste, jonka ympäri kappale kiertää ja skaalautuu. Oletuksena keskipisteessä.
transformivalikko vko 48	Valikko, jossa pivot syötetään prosentteina X, Y ja Z.

OBJ vko 49	.obj-tiedosto. 3D-tiedostomuoto. Jokainen osa alkaa rivillä, jonka alussa on o-kirjain.
virheenkorjausketju vko 49	Havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti. Projektissa tehdään kolme ketjua.
regressiotesti vko 49	Testi, joka toistaa korjatun virheen ja jää testeihin. Se varmistaa, ettei virhe palaa.
vikatehtävä vko 49	Ohjaajan tekemä tarkoituksellinen virhe. Siitä tehdään virheenkorjausketju, jos aitoa havaintoa ei ole.
JSON vko 50	tallennusmuoto. Tekstimuotoinen tiedosto, johon projektin tiedot tallennetaan ja josta ne avataan uudelleen.
tietoturva-arvio vko 50	Taulukko: uhka, testi, tulos ja toimenpide. Tässä projektissa uhkia ovat esimerkiksi haitallinen SVG ja rikottu JSON.

katselmointi vko 51	Asiakas kokeilee versiota ja kertoo, mitä muutetaan. Asiakkaan sanat kirjataan erillään omasta tulkinnasta.
haara vko 3	Oma työlinja. Muutokset eivät vaikuta päähaaraan, ennen kuin haara liitetään.
pull request vko 3	PR. Muutospyyntö, jolla haara liitetään päähaaraan. Luet muutokset ennen liittämistä.
merge vko 3	Haaran liittäminen päähaaraan.
kahva vko 5	Tartuntakohta 3D-näkymässä. Kahvasta vetämällä osaa siirretään, kierretään tai skaalataan.
Lighthouse vko 5	Chromen kehittäjätyökalujen mittari. Se mittaa sivun saavutettavuuden pisteinä 0–100.

RC vko 6	julkaisuehdokas, release candidate. Versio, jossa sisältö on jäädytetty ja joka testataan ennen v1.0:aa. Tagi on v1.0-rc1.
sisältöjäädytys vko 6	Uusia ominaisuuksia ei enää lisätä. Vain estävät virheet korjataan.
työnäyte vko 7	Yksi tuotos, joka osoittaa osaamisesi: commit, issue, testi, kuva tai muistio.
näyttömatriisi vko 7	Tämän sivun lista tutkinnon vaatimuksista. Jokaisella rivillä on linkki työnäytteeseen.

Näyttömatriisi – 31

osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **käyttää ohjelmointieditoria tai kehitysympäristöä** – VS Code, GitHub Copilot, Pythonin virtuaaliympäristö ja pytest käytössä viikoilta 40 ja 41. Versiot README:ssä. Paluuviikon ajo viikolla 2.
- **etsii ja korjaa virheitä ohjelmakoodista** – Kolme virheenkorjausketjua havaintoissueista viikoilla 49, 4 ja 7: havainto, toistamisohje, syy omin sanoin, korjauscommit, uusintatesti ja regressiotesti.
- **testaa ohjelman toimintoja** – Testitapaukset T01–T26 ja lisätestit viikoilta 41–5. Kirjoitat odotetun tuloksen itse testikoodiin ennen toteutusta. Tulokset issueissa.
- **käyttää rakenteista ohjelmointia toteutuksissa** – Kolme funktiota selityspohjalla: hierarkian muunnos (viikko 44), pivotin syötteen tarkistus (48) ja OBJ-objektijako (49). Rajapinta ennen toteutusta. Selitys ääneen seuraavan viikon palaverissa.

□ **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Hierarkiapaneeli (44), view lock (46), transformivalikko (48) sekä isot kahvat ja näppäimistökäyttö (5) käyttöliittymävaatimuksesi mukaan. Accessibility Insights ennen ja jälkeen.

□ **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – MVP-kuvauksesta tehtäväkortit ja toiminnot: tuonti (43), revolve (45), inflate (46) ja transformit (47) hyväksymiskriteerien mukaan.

□ **sopii tehtävistä tiimin muiden jäsenten kanssa** – Issue-kommentit "Sovittu viikkopalaverissa pp.kk." joka viikolta 41–7.

□ **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Revolven valintatavan vertailu ja yhteinen päätös (44–45). Osien tunnistustapa Päivitä SVG -toimintoon (3).

□ **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Katselmointi asiakkaiden kanssa (51). Palaverin arvio MVP:n ratkaisuista P1:tä varten (2).

□ **arvioi omaa toimintaa tiimin jäsenenä** – Itsearviointi kolmesta tilanteesta viikolla 9. Se lähetetään ohjaajalle Teamsissa. Päiväkirjaan kirjataan vain tilanteiden päivät ja issue-numerot.

Ohjelmistokehittäjänä toimiminen

- **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Kysymyslista asiakkaille (40). Vastaukset tiedostossa `project-docs/kysymykset.md` (41). MVP omin sanoin.
- **viestii tekniset asiat asiakaslähtöisesti** – Viiden minuutin demo katselmoinnissa (51). README ja käyttöohje käyttäjälle (6).
- **osallistuu version katselmointiin** – `project-docs/katselmointi.md`: asiakkaiden sanat, oma tulkinta ja sovitut muutokset (51).
- **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – P0/P1/P2-karsinta perusteluineen (40). Asiakkaiden prioriteetit (51). P1-järjestys palaverissa (2).
- **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Tehtäväkortit hyväksymiskriteereineen viikolta 41 alkaen. Oma rajausehdotus ja tarkistusrivi issueissa.
- **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Kaistan valinta ja toteuman vertailu Raportoi-askeleessa. Krediittimittaus (40). P1-tuntiarviot (2).
- **kehittää ohjelmiston toimintalogiikkaa** – Hierarkia, revolve, inflate, transformit ja pivot (44–48) sekä Päivitä

SVG (3).

- **valitsee ohjelmistoon sopivan tietovaraston** – Tallennustapojen vertailu oman suunnitelman kriteereillä ja oman mallin JSON-tiedoston koko (50).
- **toteuttaa yhteyden tietovarastoon** – Tallennus ja avaus JSON-tiedostona. Versionumero ja rakenteen tarkistus (50).
- **hyödyntää rajapintoja ja käsittelee tietoa** – Tiedoston avaus QFileDialogilla ja SVG:n luku svgelementsillä (43), .obj-vienti trimeshillä (49) ja JSON (50).
- **arvioi ohjelmiston tietoturvaa** – T04 haitallinen SVG (43), T22 rikottu JSON (50) ja project-docs/tietoturva.md: uhka, testi, tulos ja toimenpide.
- **käyttää versionhallintaa** – Commitit rivillä Closes #N koko projektin ajan. Tagit v0.0.1, v0.1, v1.0-rc1 ja v1.0.
- **liittää ohjelman osan olemassa olevaan versioon** – Päivitä SVG omassa haarassa, pull request ja merge päähaaraan, kun kaikki testit menevät läpi (3).
- **julkaisee ohjelman tuotantoympäristöön** – Release GitHub Actionsilla: Windows-versio, itsetesti ja zip. Harjoitus (41), MVP v0.1 (50) ja v1.0 (7).

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – PySide6 + PyVista + trimesh + pytest -pohja omin komennoin: virtuaaliympäristö, kirjastojen asennus ja tarkista_ymparisto.py (41).
- **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – project-docs/kirjastot.md: svgelements (43) ja putkigeometrian (46) rajoitteet omilla tiedostoilla kokeiltuina.
- **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – trimeshin revolve, PyVistan tube-suodatin, kamera ja orientaatiowidget sekä Qt:n transformipaneeli (45–48).
- **tuo kehittämisympäristöön ulkoisia komponentteja** – pip-paketit omin komennoin ja requirements.txt (41).
- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – MVP v0.1 komponenttikirjastolla testeineen (43–50).
- **julkaisee ohjelmiston asiakkaan ympäristöön** – MVP-release asiakkaiden kokeiltavaksi (50). Antti vahvistaa v1.0-releasen omalla Windows-koneellaan, Matti demon tai opiskelijan koneen kautta (7).

□ **dokumentoi ohjelmiston sovitulla tavalla –**

Dokumentointitapa sovittu palaverissa (43). README ja käyttöohje, joiden avulla julkaisutestaaja kulkee koko polun (6).

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.